
Voyage Assistant

Assistant IA conversationnel pour le transport

Ako Christian ■ Carl Similien ■ Noé Cervera ■ Dhanoush Kessavane

Encadrement : **Bilal Faye** (LIPN, CNRS UMR 7030)

optimalako-voyage-assistant.hf.space

L'équipe et les rôles

- **Ako Christian** — IA conversationnelle & RAG, déploiement
- **Carl Similien** — Back-end & intégration (API, base de données)
- **Noé Cervera** — Modèles & évaluation (embeddings, reranker, NER)
- **Dhanoush Kessavane** — Front-end, design & documentation

Encadrement : Bilal Faye (LIPN, CNRS UMR 7030)

Contexte et objectif

Domaine choisi : le transport de voyageurs (avion, train, bateau, bus).

- Une application **full-stack** de réservation de trajets.
- Un **assistant IA** qui répond aux questions *et* agit réellement (modification, réclamation, annulation).
- Idée directrice : **un LLM seul ne suffit pas**. Il invente et ignore les données récentes ou personnelles.
- Notre réponse : coupler le LLM à des **sources de vérité** via le RAG.

L'application est en ligne

Déployée et accessible publiquement :

[https ://optimalako-voyage-assistant.hf.space](https://optimalako-voyage-assistant.hf.space)

Recherche parmi plus de **182 000** liaisons réelles, réservation avec billet PDF et e-mail, compte Google ou e-mail, et l'assistant conversationnel.

Architecture technique

Frontend

- React 18 (sans build)
- Thèmes clair / sombre
- Voix : Web Speech API

Backend

- FastAPI + Pydantic
- SQLAlchemy (ORM)
- Services IA (LLM, RAG, NER)

Base de données

- PostgreSQL (Neon) en prod
- SQLite en local

Infrastructure

- Hugging Face Spaces (Docker)
- Un seul service : API + RAG + front

Qu'est-ce que le RAG et la vectorisation ?

RAG = *Retrieval-Augmented Generation* : on **récupère l'information pertinente avant de générer** la réponse, puis on l'injecte dans le prompt.

- **Vectorisation** : chaque texte devient un vecteur de nombres (384 dimensions) qui encode son *sens*.
- Deux textes de sens proche ont des vecteurs proches **même sans mots communs**.
- Exemple : « je veux annuler » est relié à l'article CGV « conditions de résiliation ».

Analogie : un examen « à livre ouvert » plutôt que de réciter de mémoire.

La chaîne RAG en deux étapes

1. **Indexation** : documents découpés en *chunks*, vectorisés par `gte-small` (en local), stockés dans **FAISS**.
2. **Recherche** en deux temps :
 - ▶ **Bi-encoder** : rapide, remonte 10 candidats.
 - ▶ **Cross-encoder** (*reranking*) : précis, garde les 3 meilleurs.

Plus un **NER** (`camembert-ner`) qui extrait prénom et nom d'une phrase libre.

On obtient la précision du cross-encoder sans son coût sur tout le corpus.

Les trois sources de connaissance

Selon la question, l'assistant interroge la bonne source :

- **RAG documentaire** (FAISS) — les règles : annulation, bagages. . .
- **Base de données** — les questions personnelles.
« *Combien de temps va durer mon vol ?* » → lecture du billet, calcul de la durée, réponse personnalisée.
- **APIs temps réel + web** — les données du jour : Open-Meteo (météo, heure), Frankfurter (change), DuckDuckGo.

Les infos stables (capitale, monnaie) ne sont pas appelées : le LLM les connaît.

Des actions réelles, sans hallucination

Modifier, annuler, réclamer ne sont **jamais** confiés au LLM.

Une **machine à états** déterministe pilote le parcours :

intention → vérification d'identité → **exécution réelle en base** → e-mail + billet PDF

- Le LLM **rédige** seulement ; il **n'exécute** jamais.
- Cette séparation élimine les fausses confirmations.

Sécurité

- Garde-fous : insultes, **injections de prompt**, sujets hors-périmètre.
- Anti-extraction : pas d'info sans identité vérifiée.
- SQL paramétré (anti-injection), échappement HTML de React (anti-XSS).
- Triple vérification d'identité (numéro, nom, prénom, date de naissance).
- **Aucun secret dans le code** : variables d'environnement uniquement.

Déploiement : les choix

- Le RAG charge torch + 2 modèles → **700 Mo à 1 Go de RAM**.
- Render / Railway gratuits (512 Mo) → **dépassement mémoire**.
- Choix : **Hugging Face Spaces** (16 Go gratuits, conçu pour le ML).
- Un seul service Docker : API + RAG + front (même origine, pas de CORS).
- Base **Neon** (PostgreSQL) co-localisée pour minimiser la latence.

Image Docker en torch CPU (200 Mo) ; migration des 182 k routes vers Neon.

Difficultés rencontrées et solutions

- DuckDuckGo limité → librairie ddgs (rotation multi-backend).
- Pas de date de naissance via Google → formulaire = source de vérité.
- Hallucinations d'action → machine à états séparée du LLM.
- Recherche d'escaliers à 15 s → index SQL + cache (0,34 s).
- RAG « classique » → LangChain + FAISS + reranking + NER.
- torch CUDA de 1,2 Go → version CPU dans le Docker.
- 512 Mo insuffisants → Hugging Face Spaces (16 Go).

Réponse au cahier des charges

- Domaine spécialisé : **transport**.
- Full-stack : **FastAPI + React + PostgreSQL**.
- LLM : **Gemini 2.5 Flash**.
- RAG / vectorisation : **LangChain + FAISS + gte-small + reranker**.
- Base de session : **conversations persistées**.
- Voix (option) : **Web Speech API**.
- Recherche web (bonus) : **DuckDuckGo + APIs temps réel**.
- Déploiement : **en ligne sur Hugging Face Spaces + Neon**.

Conclusion et perspectives

- Toutes les exigences de la SAE2 sont **remplies**, déploiement inclus.
- L'IA s'appuie sur des **sources de vérité** pour rester factuelle.
- Perspectives : plus de documents métier, notifications temps réel, intégration continue.

Merci de votre attention

Questions ?